

能少。

为什么会发生这些问题呢？

这是因为这个模式中的函数依赖存在某些不好的性质。这正是本章要讨论的问题。假如把这个单一的模式改造一下，分成以下三个关系模式：

```
S( { Sno, School } , { Sno → School } );
SC( { Sno, Cno, Grade } , { ( Sno, Cno ) → Grade } );
SCHOOL( { School, Mname } , { School → Mname } )
```

则这三个模式都不会发生插入异常、删除异常和更新异常的问题，数据的冗余也得到了控制。

一个模式的数据依赖会有哪些不好的性质，如何改造一个不好的模式，这就是规范化要讨论的内容。

6.2 规范化

本节首先讨论一个关系的属性间不同的依赖情况，以及如何根据属性间依赖情况判定关系是否具有某些不合适的性质。通常按属性间依赖情况区分关系的规范化程度为第一范式、第二范式、第三范式和第四范式等。然后直观地描述如何将具有不合适性质的关系转换为更合适的形式。

6.2.1 函数依赖

定义 6.1 给定关系模式 $R(U, F)$ ， X, Y 是 U 的子集。若对于 $R(U, F)$ 的任意一个可能的关系 r ， r 中不可能存在两个元组在 X 上的属性值相等，而在 Y 上的属性值不等，则称 X 函数确定 Y 或 Y 函数依赖于 X ，记作 $X \rightarrow Y$ 。

函数依赖和别的数据依赖一样是语义范畴的概念，只能根据语义来确定一个函数依赖。例如，“姓名 $\rightarrow$ 年龄”这个函数依赖只有在该部门没有同名人的条件下成立。如果允许有同名人，则年龄就不再函数依赖于姓名了。

设计者也可以对现实世界做强制性规定，例如规定不允许同名人出现，因而使“姓名 $\rightarrow$ 年龄”函数依赖成立。这样当插入某个元组时该元组上的属性值必须满足规定的函数依赖，若发现有同名人存在，则拒绝插入该元组。

注意：函数依赖不是指关系模式 R 的某个或某些关系满足的约束条件，而是指 R 在不同时刻的关系实例均要满足的约束条件。

例如，6.1 节关系模式 $\text{Student}(U, F)$ 中有 $\text{Sno} \rightarrow \text{School}$ 成立，也就是说在任何时刻该模式的关系实例（即 Student 数据表）中，不可能存在两个元组在 Sno 上的值相等，而在 School 上的值不等。因此，表 6.2 的 Student 表是错误的。因为表中有两个元组在 Sno 上都等于“S1”，而

在 School 上一个为“信息学院”，一个为“人工智能学院”。

表 6.2 一个错误的 Student 表

Sno	School	Mname	Cno	Grade
S1	信息学院	张明	C1	95
S1	人工智能学院	张明	C1	90
S3	信息学院	张明	C1	88
S4	信息学院	张明	C1	70
S5	信息学院	张明	C1	78
⋮	⋮	⋮	⋮	⋮

下面介绍一些术语和记号。

① $X \rightarrow Y$, 但 $Y \not\subseteq X$, 则称 $X \rightarrow Y$ 是非平凡的函数依赖(non-trivial functional dependency)。

② $X \rightarrow Y$, 但 $Y \subseteq X$, 则称 $X \rightarrow Y$ 是平凡的函数依赖(trivial functional dependency)。对于任一关系模式, 平凡的函数依赖都是必然成立的, 它不反映新的语义。若不特别声明, 总是讨论非平凡的函数依赖。

③ 若 $X \rightarrow Y$, 则 X 称为这个函数依赖的决定属性组, 也称为决定因素(determinant)。

④ 若 $X \rightarrow Y, Y \rightarrow X$, 则记作 $X \leftrightarrow Y$ 。

⑤ 若 Y 不函数依赖于 X , 则记作 $X \nrightarrow Y$ 。

定义 6.2 在 $R(U, F)$ 中, 如果 $X \rightarrow Y$, 并且对于 X 的任何一个真子集 X' , 都有 $X' \nrightarrow Y$, 则称 Y 对 X 完全函数依赖(full functional dependency), 记作 $X \xrightarrow{F} Y$ 。

若 $X \rightarrow Y$, 但 Y 不完全函数依赖于 X , 则称 Y 对 X 部分函数依赖(partial functional dependency), 记作 $X \xrightarrow{P} Y$ 。

例 6.1 中 $(Sno, Cno) \xrightarrow{F} Grade$ 是完全函数依赖, $(Sno, Cno) \xrightarrow{P} School$ 是部分函数依赖, 因为 $Sno \rightarrow School$ 成立, 而 Sno 是 (Sno, Cno) 的真子集。

定义 6.3 在 $R(U, F)$ 中, 如果 $X \rightarrow Y (Y \not\subseteq X)$, $Y \nrightarrow X$, $Y \rightarrow Z$, $Z \not\subseteq Y$, 则称 Z 对 X 传递函数依赖(transitive functional dependency)。记为 $X \xrightarrow{T} Z$ 。

这里加上条件 $Y \nrightarrow X$, 是因为如果 $Y \rightarrow X$, 则 $X \leftrightarrow Y$, 实际上是直接函数依赖而不是传递函数依赖。

例 6.1 中有 $Sno \rightarrow School$, $School \rightarrow Mname$ 成立, 所以 $Sno \xrightarrow{T} Mname$ 。

6.2.2 码^①

码是关系模式中的一个重要概念。在第2章2.1节已给出了有关码的若干描述性定义，这里用函数依赖的概念来定义码。

定义 6.4 设 K 为 $R(U, F)$ 中的属性或属性组，若 $K \xrightarrow{F} U$ ，则 K 为 R 的候选码。

注意 U 是完全函数依赖于 K ，而不是部分函数依赖于 K 。

如果 U 部分函数依赖于 K ，即 $K \xrightarrow{P} U$ ，则 K 称为超码。

候选码和超码有什么关系呢？候选码的超集（如果存在）一定是超码，候选码的任何真子集一定不是超码。

若候选码多于一个，则选定其中的一个为主码。

包含在任何一个候选码中的属性称为主属性，不包含在任何一个候选码中的属性称为非主属性或非码属性。最简单的情况，单个属性是码；最极端的情况，整个属性组是码，称为全码。

在后面的章节中主码或候选码都简称为码。

[例 6.2] 第2章的“学生”表 $Student(\underline{Sno}, Sname, Ssex, Sbirthdate, Smajor)$ 中，单个属性“ $\underline{Sno}$ ”是码，用下划线显示出来。“学生选课”表 $SC(\underline{Sno}, \underline{Cno}, Grade, Semester, Teachingclass)$ 中，属性组“ $\underline{Sno}, \underline{Cno}$ ”是码。

[例 6.3] 关系模式 $R(P, W, A)$ 中，属性 P 表示演奏者， W 表示作品， A 表示听众。假设一个演奏者可以演奏多部作品，某一部作品可被多个演奏者演奏，听众也可以欣赏不同演奏者的不同作品，这个关系模式的码为 (P, W, A) ，即全码。

定义 6.5 关系模式 R 中属性或属性组 X 并非 R 的码，但 X 是另一个关系模式的码，则称 X 是 R 的外码。

如第2章的“学生选课”表 $SC(\underline{Sno}, \underline{Cno}, Grade, Semester, Teachingclass)$ 中， Sno 不是码，但 Sno 是关系模式 $Student(\underline{Sno}, Sname, Ssex, Sbirthdate, Smajor)$ 的码，则 Sno 是关系模式 SC 的外码。主码与外码提供了一个表示关系间联系的手段。

6.2.3 范式

关系数据库中的关系是要满足一定要求的，满足不同程度要求的关系为不同范式。满足最低要求的关系叫第一范式，简称 1NF；在第一范式中满足进一步要求的关系为第二范式，其余以此类推。

有关范式理论的研究主要是 Edgar F. Codd 做的工作。1971—1972 年 Codd 系统地提出了 1NF、2NF、3NF 的概念，讨论了规范化的问题。1974 年，Codd 和 Boyce 共同提出了一个新范

^① 在一些教材和文章中也称码为“键”或“键码”。

式, 即 BCNF。1976 年 Fagin 提出了 4NF。

所谓“第几范式”原本是表示关系的某一种级别, 所以常称某一关系模式 R 为第几范式。现在则把范式这个概念理解成符合某一种级别的关系模式的集合, 即 R 为第几范式就可以写成 $R \in xNF$ 。

对于各种范式之间的关系有

$$4NF \subset BCNF \subset 3NF \subset 2NF \subset 1NF$$

成立, 如图 6.2 所示。

一个低一级范式的关系模式通过模式分解 (schema decomposition) 可以转换为若干个更高级别范式的关系模式的集合, 这种过程就叫规范化。

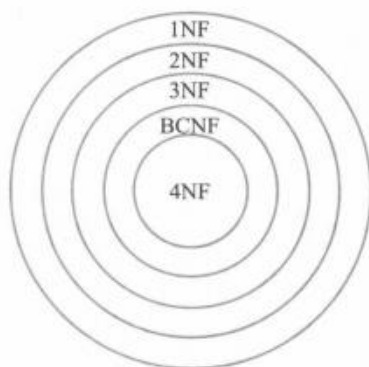


图 6.2 各种范式之间的关系

6.2.4 2NF

如果关系模式 R 的每一个分量是一个不可分的数据项, R 是满足最低要求的关系模式, 则 $R \in 1NF$ 。

定义 6.6 若 $R \in 1NF$, 且每一个非主属性完全函数依赖于任何一个候选码, 则 $R \in 2NF$ 。

下面举一个不是 2NF 的例子。

[例 6.4] 有关系模式 $S-L-C(Sno, School, Sloc, Cno, Grade)$, 其中 $Sloc$ 为学生的住所, 并且每个学院的学生住在同一个宿舍楼。 $S-L-C$ 的码为 (Sno, Cno) 。则函数依赖有

$$\begin{aligned} (Sno, Cno) &\xrightarrow{F} Grade \\ Sno &\rightarrow School, (Sno, Cno) \xrightarrow{P} School \\ Sno &\rightarrow Sloc, (Sno, Cno) \xrightarrow{P} Sloc, \\ School &\rightarrow Sloc \text{ (每个学院的学生住同一个宿舍楼)} \end{aligned}$$

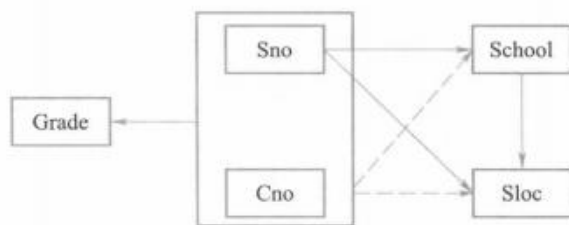


图 6.3 函数依赖示例

函数依赖关系如图 6.3 所示。

图中用虚线表示部分函数依赖。另外, $School$ 函数确定 $Sloc$, 这一点在讨论第二范式时暂不考虑。可以看到, 非主属性 $School$ 、 $Sloc$ 并不完全函数依赖于码。因此 $S-L-C(Sno, School, Sloc, Cno, Grade)$ 不符合 2NF 定义, 即 $S-L-C \notin 2NF$ 。

一个关系模式 R 不属于 2NF, 就会产生以下几个问题:

① 插入异常。假若要插入一个学生 $Sno=S7$, $School=INF$, $Sloc=BLD2$, 但该生还未选课, 即这个学生无 Cno , 这样的元组就插不进 $S-L-C$ 中。因为插入元组时必须给定码值, 而这时码值的一部分为空, 因而学生的固有信息无法插入。

② 删除异常。假定某个学生只选一门课，如 S4 就选了一门课 C3，现在 C3 这门课他也不选了，那么 C3 这个数据项就要删除。而 C3 是主属性，删除了 C3，整个元组就必须一起删除，使得 S4 的其他信息也被删除了，从而造成删除异常，即不应删除的信息也删除了。

③ 修改复杂。某个学生从人工智能学院转到信息学院，这本来只需修改此学生元组中的 School 分量即可，但因为关系模式 S-L-C 中还含有学生的住所 Sloc 属性，学生转学院将同时改变住所，因而还必须修改元组中的 Sloc 分量。另外，如果这个学生选修了 k 门课，School、Sloc 重复存储了 k 次，不仅存储冗余度大，而且必须无遗漏地修改 k 个元组中的全部 School、Sloc 信息，造成修改的复杂化。

分析上面的例子可以发现问题在于有两类非主属性，一类如 Grade，它对码是完全函数依赖；另一类如 School、Sloc，它们对码不是完全函数依赖。解决的办法是用投影分解把关系模式 S-L-C 分解为两个关系模式：SC(Sno, Cno, Grade) 和 S-L(Sno, School, Sloc)。

关系模式 SC 与 S-L 中属性间的函数依赖可以用图 6.4 和图 6.5 表示如下。

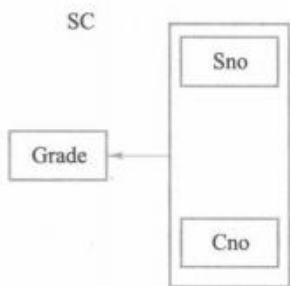


图 6.4 SC 中的函数依赖

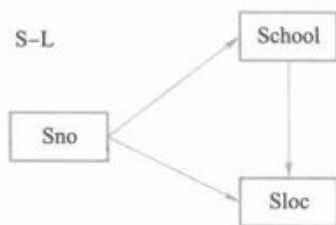


图 6.5 S-L 中的函数依赖

关系模式 SC 的码为 (Sno, Cno)，关系模式 S-L 的码为 Sno，这样就使得非主属性对码都是完全函数依赖了。

6.2.5 3NF

定义 6.7 设关系模式 $R(U, F) \in 1NF$ ，若 R 中不存在这样的码 X ，属性组 Y 及非主属性 $Z (Y \not\subseteq Z)$ ，使得 $X \rightarrow Y$ ， $Y \rightarrow Z$ 成立， $Y \not\rightarrow X$ ，则称 $R(U, F) \in 3NF$ 。

由定义 6.7 可以证明，若 $R \in 3NF$ ，则每一个非主属性既不传递依赖于码，也不部分依赖于码。也就是说，可以证明如果 R 属于 3NF，则必有 R 属于 2NF。

在图 6.4 中关系模式 SC 没有传递依赖，而图 6.5 中关系模式 S-L 存在非主属性对码的传递依赖。在 S-L 中，由 $Sno \rightarrow School$ ($School \not\rightarrow Sno$)， $School \rightarrow Sloc$ ，可得 $Sno \xrightarrow{T} Sloc$ 。

因此 $SC \in 3NF$ ，而 $S-L \notin 3NF$ 。

一个关系模式 R 若不是 3NF, 就会产生与 6.2.4 节中 2NF 相类似的问题。读者可以类比 2NF 的反例加以说明。

解决的办法同样是将 $S-L$ 分解为 $S-SS(Sno, School)$ 和 $S-SL(School, Sloc)$ 。分解后的关系模式 $S-SS$ 与 $S-SL$ 中不再存在传递依赖。

6.2.6 BCNF

BCNF(Boyce Codd Normal Form)是由 Boyce 与 Codd 提出的, 比上述 3NF 又进了一步, 通常认为 BCNF 是修正的第三范式, 有时也称为扩充的第三范式。

定义 6.8 关系模式 $R(U, F) \in 1NF$, 若 $X \rightarrow Y$ 且 $Y \not\subseteq X$ 时 X 必含有码, 则 $R(U, F) \in BCNF$ 。也就是说, 关系模式 $R(U, F)$ 中, 若每一个决定因素都包含码, 则 $R(U, F) \in BCNF$ 。

由 BCNF 的定义可以得到结论, 一个满足 BCNF 的关系模式有:

- ① 所有非主属性对每一个码都是完全函数依赖。
- ② 所有主属性对每一个不包含它的码也是完全函数依赖。
- ③ 没有任何属性完全函数依赖于非码的任何一组属性。

由于 $R \in BCNF$, 按定义排除了任何属性对码的传递依赖与部分依赖, 所以 $R \in 3NF$ 。严格的证明留给读者完成。但是若 $R \in 3NF$, R 未必属于 BCNF。

下面用几个例子说明属于 3NF 的关系模式有的属于 BCNF, 但有的不属于 BCNF。

[例 6.5] 考察关系模式 $Course(\underline{Cno}, Cname, Ccredit, Cpro)$, 它只有一个码 Cno , 这里没有任何属性对 Cno 部分依赖或传递依赖, 所以 $Course \in 3NF$ 。同时 $Course$ 中 Cno 是唯一的决定因素, 所以 $Course \in BCNF$ 。

对于关系模式 $SC(\underline{Sno}, \underline{Cno}, Grade, Semester, Teachingclass)$ 可做同样分析。

[例 6.6] 关系模式 $Student(\underline{Sno}, Sname, Ssex, Sbirthdate, Smajor)$ 中, 假定 $Sname$ 也具有唯一性, 那么 $Student$ 就有两个候选码, 这两个码都由单个属性组成, 彼此不相交。其他属性不存在对码的传递依赖与部分依赖, 所以 $Student \in 3NF$ 。同时 $Student$ 中除 Sno 、 $Sname$ 外没有其他决定因素, 所以 $Student \in BCNF$ 。

以下再举几个例子。

[例 6.7] 关系模式 $SJP(S, J, P)$ 中, S 是学生, J 表示课程, P 表示名次。每一个学生选修每门课程的成绩有一定的名次, 每门课程中每一名次只有一个学生(即没有并列名次)。由语义可得到下面的函数依赖:

$$(S, J) \rightarrow P; (J, P) \rightarrow S$$

所以 (S, J) 与 (J, P) 都可以作为候选码。这两个码各由两个属性组成, 而且它们是相交的。这个关系模式中显然没有属性对码传递依赖或部分依赖, 所以 $SJP \in 3NF$, 而且除 (S, J) 与 (J, P) 以外没有其他决定因素, 所以 $SJP \in BCNF$ 。

[例 6.8] 关系模式 $STJ(S, T, J)$ 中, S 表示学生, T 表示教师, J 表示课程。每一教师只教一门课, 每门课有若干教师, 某一学生选定某门课就对应一个固定的教师。由语义可得到如下的函数依赖:

$$(S, J) \rightarrow T, (S, T) \rightarrow J, T \rightarrow J$$

函数依赖关系可以用图 6.6 表示, 这里 (S, J) 、 (S, T) 都是候选码。

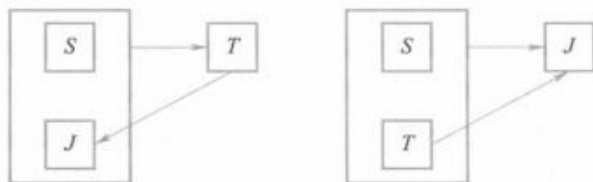


图 6.6 STJ 中的函数依赖

STJ 是 3NF, 因为没有任何非主属性对码传递依赖或部分依赖, 但 STJ 不是 BCNF 关系, 因为 T 是决定因素, 而 T 不包含码。

对于不是 BCNF 的关系模式, 仍然存在不合适的地方。读者可自己举例指出 STJ 的不合适之处。非 BCNF 的关系模式也可以通过分解成为 BCNF。例如 STJ 可分解为 $ST(S, T)$ 与 $TJ(T, J)$, 它们都是 BCNF。

3NF 和 BCNF 是在函数依赖的条件下对模式分解所能达到的分离程度的测度。一个模式中的关系模式如果都属于 BCNF, 那么在函数依赖范畴内它已实现了彻底分离, 已消除了插入和删除的异常。3NF 的“不彻底”性表现在可能存在主属性对码的部分依赖和传递依赖。

6.2.7 多值依赖

以上是在函数依赖的范畴内讨论问题。属于 BCNF 的关系模式是否就很完美了呢? 下面讲解多值依赖。首先来看一个例子。

[例 6.9] 学校中如果某一门课程由多个教师讲授, 他们使用一套相同的参考书。每个教师可以讲授多门课程, 每种参考书可以供多门课程使用。可以用一个非规范化的关系来描述课程 C 、教师 T 和参考书 B 之间的关系, 如表 6.3 所示。

把这张表变成一张规范化的二维表 Teaching, 如表 6.4 所示。

关系模型 Teaching($\underline{C}, T, B$) 的码是 ($\underline{C}, T, B$), 即全码, 因而 Teaching $\in$ BCNF。但是当某一课程(如物理)增加一名讲课教师(如周英)时, 必须插入多个(这里是三个)元组: (物理, 周英, 普通物理学), (物理, 周英, 光学原理), (物理, 周英, 物理习题集)。

同样, 某一门课程(如数学)要去掉一本参考书(如微分方程), 则必须删除多个(这里是两个)元组: (数学, 李勇, 微分方程), (数学, 张平, 微分方程)。

因而对数据的增、删、改很不方便, 数据的冗余也十分明显。仔细考察这类关系模式, 发现它具有一种称之为多值依赖的数据依赖。

表 6.3 非规范化关系示例

课程 C	教师 T	参考书 B
物理	{李勇 王军}	{普通物理学 光学原理 物理习题集}
数学	{李勇 张平}	{数学分析 微分方程 高等代数}
计算数学	{张平 周峰}	{数学分析}
$\vdots$	$\vdots$	$\vdots$

表 6.4 规范化的二维表 Teaching

课程 C	教师 T	参考书 B
物理	李勇	普通物理学
物理	李勇	光学原理
物理	李勇	物理习题集
物理	王军	普通物理学
物理	王军	光学原理
物理	王军	物理习题集
数学	李勇	数学分析
数学	李勇	微分方程
数学	李勇	高等代数
数学	张平	数学分析
数学	张平	微分方程
数学	张平	高等代数
$\vdots$	$\vdots$	$\vdots$

定义 6.9 给定关系模式 $R(U, F)$, X, Y, Z 是 U 的子集, 并且 $Z = U - X - Y$ 。关系模式中多值依赖 $X \twoheadrightarrow Y$ 成立, 当且仅当对 $R(U, F)$ 的任一关系 r 给定的一对 (x, z) 值, 有一组 Y 的值, 这组值仅仅决定于 x 值而与 z 值无关。

例如, 在表 6.4 的关系模式 Teaching 中, 对于一个 (物理, 光学原理) 有一组 T 值 {李勇, 王军}, 这组值仅仅决定于课程 C 上的值 (物理)。也就是说, 对于另一个 (物理, 普通物理学), 它对应的一组 T 值仍是 {李勇, 王军}, 尽管这时参考书 B 的值已经改变了。因此 T 多值依赖于 C , 即 $C \twoheadrightarrow T$ 。

对于多值依赖的另一个等价的形式化定义是: 在 $R(U, F)$ 的任一关系 r 中, 如果存在元组 t, s , 使得 $t[X] = s[X]$, 那么就必然存在元组 $w, v \in r$ (w, v 可以与 s, t 相同), 使得 $w[X] = v[X] = t[X]$, 而 $w[Y] = t[Y]$, $w[Z] = s[Z]$, $v[Y] = s[Y]$, $v[Z] = t[Z]$ (即交换 s, t 元组的 Y 值所得的两个新元组必在 r 中), 则 Y 多值依赖于 X , 记为 $X \twoheadrightarrow Y$ 。这里, X, Y 是 U 的子集, $Z = U - X - Y$ 。

若 $X \twoheadrightarrow Y$, 而 $Z = \phi$, 即 Z 为空, 则称 $X \twoheadrightarrow Y$ 为平凡的多值依赖。即对于 $R(X, Y)$, 如果有 $X \twoheadrightarrow Y$ 成立, 则 $X \twoheadrightarrow Y$ 为平凡的多值依赖。

下面再举一个具有多值依赖的关系模式的例子。

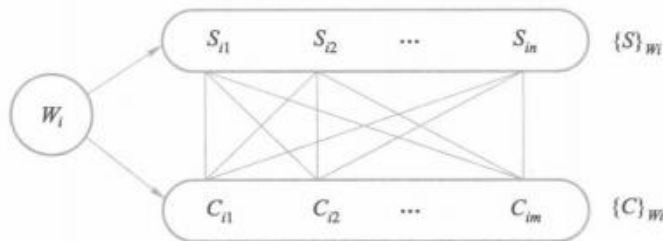
[例 6.10] 关系模式 WSC(W, S, C) 中, W 表示仓库, S 表示保管员, C 表示商品。假设每个仓库有若干个保管员和若干种商品, 每个保管员保管所在仓库的所有商品, 每种商品被所有保管员保管。列出关系如表 6.5 所示。

按照语义, 对于 W 的每一个值 W_i , S 有一个完整的集合与之对应而不问 C 的取何值, 所以 $W \twoheadrightarrow S$ 。

如果用图 6.7 来表示这种对应, 则对应 W 的某一个值 W_i 的全部 S 值记作 $\{S\}_{W_i}$ (表示此仓库工作的全部保管员), 全部 C 值记作 $\{C\}_{W_i}$ (表示在此仓库中存放的所有商品)。应当有 $\{S\}_{W_i}$ 中的每一个值和 $\{C\}_{W_i}$ 中的每一个 C 值对应。于是 $\{S\}_{W_i}$ 与 $\{C\}_{W_i}$ 之间正好形成一个完全二分图, 因而 $W \twoheadrightarrow S$ 。

表 6.5 WSC 表

W	S	C
W_1	S_1	C_1
W_1	S_1	C_2
W_1	S_1	C_3
W_1	S_2	C_1
W_1	S_2	C_2
W_1	S_2	C_3
W_2	S_3	C_4
W_2	S_3	C_5
W_2	S_4	C_4
W_2	S_4	C_5

图 6.7 $W \twoheadrightarrow S$ 且 $W \twoheadrightarrow C$

直观上, 一个关系模式中的两个属性如果彼此之间没有任何联系, 则它们是相互独立的。但如果它们都和一个共同的属性有关, 则将这两个无关的属性放在一个关系中时会造成存储的冗余。在上面的这个例子中, 保管员 S 和商品 C 是相互独立的, 但是它们都和仓库 W 有联系, W 确定了, 则有一组保管员和一批商品与之对应。但是每一个保管员需要管理该仓库中的所有商品, 换言之, 保管员和商品之间在确定的仓库下是完全二分图的联系, 表达这样的联系会浪费存储空间, 没有实际的意义。在这样的情况下, 将其分离存储很有必要。也就是, 把 W 和 S 组成一个关系, W 和 C 组成另一个关系。

由于 C 与 S 的完全对称性, 必然有 $W \twoheadrightarrow C$ 成立。

多值依赖具有以下性质:

① 多值依赖具有对称性。即若 $X \twoheadrightarrow Y$, 则 $X \twoheadrightarrow Z$, 其中 $Z = U - X - Y$ 。

从例 6.10 容易看出, 因为每个保管员保管所有商品, 同时每种商品被所有保管员保管, 显然若 $W \twoheadrightarrow S$, 必然有 $W \twoheadrightarrow C$ 。

② 多值依赖具有传递性。即如果 $XYZ = U$, $X \twoheadrightarrow Y$, $Y \twoheadrightarrow Z$, 则 $X \twoheadrightarrow Z - Y$ 。

③ 函数依赖可以看作是多值依赖的特殊情况, 即若 $X \rightarrow Y$, 则 $X \twoheadrightarrow Y$ 。这是因为当 $X \rightarrow Y$ 时, 对 X 的每一个值 x , Y 有一个确定的值 y 与之对应, 故 $X \twoheadrightarrow Y$ 。

④ 若 $X \twoheadrightarrow Y$, $X \twoheadrightarrow Z$, 则 $X \twoheadrightarrow YZ$ 。

⑤ 若 $X \twoheadrightarrow Y$, $X \twoheadrightarrow Z$, 则 $X \twoheadrightarrow Y \cap Z$ 。

⑥ 若 $X \twoheadrightarrow Y$, $X \twoheadrightarrow Z$, 则 $X \twoheadrightarrow Y-Z$, $X \twoheadrightarrow Z-Y$ 。

多值依赖与函数依赖相比, 具有下面两个基本的区别:

① 多值依赖的有效性与属性组的范围有关。若 $X \twoheadrightarrow Y$ 在 U 上成立, 则在 $W (XY \subseteq W \subseteq U)$ 上一定成立; 反之则不然, 即 $X \twoheadrightarrow Y$ 在 $W (W \subset U)$ 上成立, 在 U 上并不一定成立。这是因为多值依赖的定义中不仅涉及 U 的子集 X 和 Y , 而且还涉及 U 中其余的子集 Z 。

一般地, 在 $R(U, F)$ 上若有 $X \twoheadrightarrow Y$ 在 $W (W \subset U)$ 上成立, 则称 $X \twoheadrightarrow Y$ 为 $R(U, F)$ 的嵌入型多值依赖。

但是在关系模式 $R(U, F)$ 中, 函数依赖 $X \rightarrow Y$ 的有效性仅决定于 X 、 Y 的值。只要在 $R(U, F)$ 的任何一个关系 r 中, 元组在 X 和 Y 上的值满足定义 6.1, 则函数依赖 $X \rightarrow Y$ 在任何属性组 $W (XY \subseteq W \subseteq U)$ 上成立。

② 若函数依赖 $X \rightarrow Y$ 在 $R(U, F)$ 上成立, 则对于任何 $Y' \subset Y$ 均有 $X \rightarrow Y'$ 成立。而多值依赖 $X \twoheadrightarrow Y$ 若在 $R(U, F)$ 上成立, 却不能断言对于任何 $Y' \subset Y$ 有 $X \twoheadrightarrow Y'$ 成立。

例如, 有关系 $R(A, B, C, D)$, $A \twoheadrightarrow BC$, 当然也有 $A \twoheadrightarrow D$ 成立。有 R 的一个关系实例, 在此实例上 $A \twoheadrightarrow B$ 是不成立的, 如表 6.6 所示。

表 6.6 R 的一个实例

A	B	C	D
a_1	b_1	c_1	d_1
a_1	b_1	c_1	d_2
a_1	b_2	c_2	d_1
a_1	b_2	c_2	d_2

6.2.8 4NF

定义 6.10 关系模式 $R(U, F) \in 1NF$, 如果对于 R 的每个非平凡多值依赖 $X \twoheadrightarrow Y (Y \not\subseteq X)$, X 都含有码, 则称 $R(U, F) \in 4NF$ 。

4NF 就是限制关系模式的属性之间不允许有非平凡且非函数依赖的多值依赖。因为根据定义, 对于每一个非平凡的多值依赖 $X \twoheadrightarrow Y$, X 都含有候选码, 于是就有 $X \rightarrow Y$, 所以 4NF 所允许的非平凡的多值依赖实际上是函数依赖。

显然, 如果一个关系模式是 4NF, 则必为 BCNF。

在前面讨论的关系模式 WSC 中, $W \twoheadrightarrow S$, $W \twoheadrightarrow C$, 它们都是非平凡的多值依赖。而 W 不是码, 关系模式 WSC 的码是 $(\underline{W}, S, C)$, 即全码。因此 $WSC \notin 4NF$ 。

一个关系模式如果已达到了 BCNF 但不是 4NF, 这样的关系模式仍然具有不好的性质。以 WSC 为例, $WSC \notin 4NF$, 但是 $WSC \in BCNF$ 。对于 WSC 的某个关系, 若某一仓库 W_i 有 n 个保管员, 存放 m 件物品, 则关系中分量为 W_i 的元组数目一定有 $m \times n$ 个。每个保管员重复存储 m 次, 每种物品重复存储 n 次, 数据的冗余度太大, 因此还应该继续规范化使关系模式 WSC 达到 4NF。

可以用投影分解的方法消去非平凡且非函数依赖的多值依赖。例如,可以把WSC分解为 $WS(W,S)$, $WC(W,C)$ 。在WS中虽然有 $W \twoheadrightarrow S$,但这是平凡的多值依赖。WS中已不存在非平凡的非函数依赖的多值依赖,所以 $WS \in 4NF$,同理 $WC \in 4NF$ 。

函数依赖和多值依赖是两种最重要的数据依赖。如果只考虑函数依赖,则属于BCNF的关系模式规范化程度已经是最高了;如果考虑多值依赖,则属于4NF的关系模式规范化程度是最高的。事实上,数据依赖中除函数依赖和多值依赖之外,还有其他数据依赖,例如有一种连接依赖。函数依赖是多值依赖的一种特殊情况,而多值依赖实际上又是连接依赖的一种特殊情况。但连接依赖不像函数依赖和多值依赖一样可由语义直接导出,而是在关系的连接运算时才反映出来。存在连接依赖的关系模式仍可能遇到数据冗余及插入、修改、删除异常等问题。如果消除了属于4NF的关系模式中存在的连接依赖,则可以进一步达到5NF的关系模式。这里不再讨论连接依赖和5NF,有兴趣的读者可以参阅有关书籍。

规范化小结

在关系数据库中,对关系模式的基本要求是满足第一范式,这样的关系模式就是合法的、允许的。但是,人们发现有些关系模式存在插入、删除异常,以及修改复杂、数据冗余等问题,需要寻求解决这些问题的方法,这就是规范化的目的。

规范化的基本思想是逐步消除数据依赖中不合适的部分,使模式中的各关系模式达到某种程度的“分离”,即“一事一地”的模式设计原则。让一个关系描述一个概念、一个实体或者实体间的一种联系。若多于一个概念就把它“分离”出去。因此,所谓规范化实质上是概念的单一化。

人们认识这个原则是经历了一个过程的。从认识非主属性的部分函数依赖的危害开始,2NF、3NF、BCNF、4NF的相继提出是这个认识过程逐步深化的标志,图6.8可以概括这个过程。

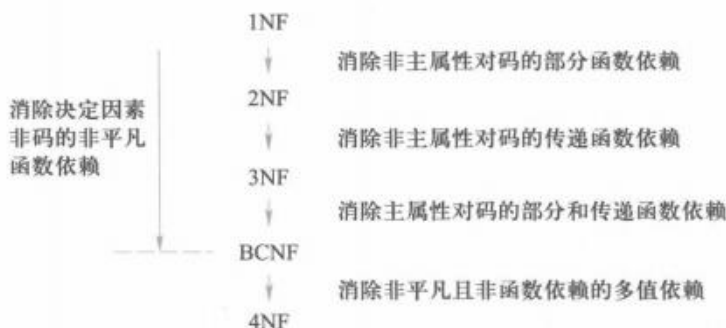


图 6.8 规范化过程

关系模式的规范化过程是通过对关系模式的分解来实现的,即把低一级的关系模式分解为若干个更高级别的关系模式。因此,关系规范化程度可以自然地用于设计“好”的关系模式标准,规范化程度越高,可能的数据异常就越少。

6.3 数据依赖的公理系统

在明确了设计标准以后,紧接着一个问题就是,是否可以找到一个算法,通过分解模式来获得达到某个规范化程度(比如 3NF)的模式集合呢?如果存在这样的算法,就可以开发出关系模式的自动化设计工具,以减轻设计人员的负担。

数据依赖的公理系统是模式分解算法的理论基础。下面首先讨论函数依赖的一个有效而完备的公理系统——Armstrong 公理系统。

定义 6.11 给定关系模式 $R(U, F)$, 其任何一个关系 r , 若函数依赖 $X \rightarrow Y$ 都成立(即 r 中任意两元组 t, s , 若 $t[X] = s[X]$, 则 $t[Y] = s[Y]$), 则称 F 逻辑蕴涵 $X \rightarrow Y$ 。

为了从一组函数依赖求得蕴涵的函数依赖,例如已知函数依赖集 F , 要问 $X \rightarrow Y$ 是否为 F 所蕴涵,就需要一套推理规则,这组推理规则是 1974 年首先由 Armstrong 提出来的。

Armstrong 公理系统(Armstrong's axiom) 设 U 为属性组全集, F 是 U 上的一组函数依赖,于是有关系模式 $R(U, F)$, 对 $R(U, F)$ 来说有以下的推理规则。

A1 自反律(reflexivity rule): 若 $Y \subseteq X \subseteq U$, 则 $X \rightarrow Y$ 为 F 所蕴涵。

A2 增广律(augmentation rule): 若 $X \rightarrow Y$ 为 F 所蕴涵, 且 $Z \subseteq U$, 则 $XZ \rightarrow YZ$ ^① 为 F 所蕴涵。

A3 传递律(transitivity rule): 若 $X \rightarrow Y$ 及 $Y \rightarrow Z$ 为 F 所蕴涵, 则 $X \rightarrow Z$ 为 F 所蕴涵。

注意: 由自反律所得到的函数依赖均是平凡的函数依赖, 自反律的使用并不依赖于 F 。

定理 6.1 Armstrong 推理规则是正确的。

下面从定义出发证明推理规则的正确性。

证

(1) 设 $Y \subseteq X \subseteq U$

对 $R(U, F)$ 的任一关系 r 中的任意两个元组 t, s :

若 $t[X] = s[X]$, 由于 $Y \subseteq X$, 有 $t[Y] = s[Y]$,

所以 $X \rightarrow Y$ 成立, 自反律得证^②。

(2) 设 $X \rightarrow Y$ 为 F 所蕴涵, 且 $Z \subseteq U$

设 $R(U, F)$ 的任一关系 r 中的任意两个元组 t, s :

若 $t[XZ] = s[XZ]$, 则有 $t[X] = s[X]$ 和 $t[Z] = s[Z]$;

由 $X \rightarrow Y$, 于是有 $t[Y] = s[Y]$, 所以 $t[YZ] = s[YZ]$, $XZ \rightarrow YZ$ 为 F 所蕴涵, 增广律得证。

① 为了简单起见, 用 XZ 代表 $X \cup Z$, YZ 代表 $Y \cup Z$ 。

② $t[X]$ 表示元组 t 在属性(组) X 上的分量, 等价于 $t.X$ 。